WebSDR-Digital-Receive-Only

Receive-only digital-mode stations fed by a web SDR or KiwiSDR

JS8Call + CommStat · FLDIGI + FLMSG + FLAMP

```
web SDR / KiwiSDR -> browser tab -> virtual audio device -> JS8Call -> CommStat
web SDR / KiwiSDR -> browser tab -> virtual audio device -> FLDIGI  -> FLMSG / FLAMP

No radio. No antenna. No transmitting.
```

Version 0.2.0 · October 2026

KC3WHJ

https://github.com/KC3WHJ/WebSDR-Digital-Receive-Only

Generated from the project's README and docs by docs/build-guide.py.

Contents

Overview 4

What you get 4

Receive-only, by design 4

Requirements 5

Quick start 5

The prerequisite check 6

What gets installed, and where 6

Uninstall 6

Things worth knowing 7

Testing 7

Part of a bigger station 8

Credits and licenses 8

Using it 9

JS8Call + CommStat: a session, start to finish 9

Tuning the web SDR 9

CommStat 9

Options 10

Changing settings 10

FLDIGI + FLMSG + FLAMP: a session, start to finish 10

How it works 12

The audio path 12

Why a separate JS8Call profile 12

The receive-only guard 12

Startup order and detaching 12

Virtual devices: never duplicated, always fully removed 13

The installer 13

Layout 13

The FLDIGI station 14

Troubleshooting 15

The installer stops with FAIL items 15

JS8Call shows nothing / no audio 15

The browser goes silent after I finish 15

CommStat can't connect / its connector says Disabled 16

CommStat's map is blank 16
"receive-only guard: ..." and it won't start 16
The JS8Call window title says "de KN4CRD" 16
An old install left a silent audio device behind 16
Updating 16
FLDIGI shows nothing / a flat waterfall 16
FLMSG or FLAMP doesn't open, or says it can't reach FLDIGI 17
FLDIGI's settings changes don't stick 17

Overview

Receive-only digital-mode stations, fed by any web SDR or KiwiSDR. No radio, no antenna, no transmitting: play a WebSDR in your browser, and this sets up the decoders to listen to it:

- **JS8Call + CommStat** - JS8Call decodes JS8 traffic and CommStat shows what it hears (map, STATREPs).
- **FLDIGI + FLMSG + FLAMP** - FLDIGI decodes PSK, MFSK, Contestia, Olivia, RTTY and more; FLMSG opens received forms and FLAMP reassembles received files (NBEMS traffic, e.g. AmRRON nets).

It is a small installer and a set of launchers - not a copy of any of these programs. It installs them from your distribution (keeping any you already have), pulls CommStat from its own GitHub repository, and configures everything for listening only.

```
web SDR / KiwiSDR  ->  browser tab  ->  virtual audio device  ->  JS8Call  ->  CommStat
                                          (receive only)          (no rig)     (map, STATREPs)

web SDR / KiwiSDR  ->  browser tab  ->  virtual audio device  ->  FLDIGI  ->  FLMSG / FLAMP
                                          (receive only)          (no rig)    (forms, files)
```

The two stations are independent: run either, or both at once (two browser tabs on two frequencies).

*This project was called **JS8Call-CommStat-Receive-Only** before the FLDIGI station was added. GitHub redirects the old address, and existing installs keep working.*

What you get

- A **prerequisite check** that runs before anything is installed, and refuses to touch your system if a hard requirement isn't met (`./preflight.sh`).
- One installer (`./install.sh`) and launchers for each station: `websdr-js8-start` / `websdr-js8-stop` and `websdr-fldigi-start` / `websdr-fldigi-stop` (also in your applications menu and on the Desktop).
- **Separate settings for everything**: a JS8Call profile, a CommStat copy, and FLDIGI/FLMSG/FLAMP settings folders of their own, so a normal radio setup you may already have is never disturbed.
- A guided step that routes the browser tab playing the WebSDR into the decoder.
- **The programs detach from the terminal**, so closing the launcher's window doesn't kill them.

Receive-only, by design

Nothing here can key a radio, because there isn't one to key. On top of that:

Safeguard	What it does
No rig control	The JS8Call profile has `Rig=None` and no CAT or PTT port. FLDIGI's settings have Hamlib, RigCAT and flrig control and **every PTT method** (CAT, serial RTS/DTR, GPIO, parallel port, CM108, right-channel tone) switched off.

Safeguard	What it does
Start-up guard	`websdr-js8-start` re-checks the JS8Call profile every time and **refuses to start** if rig control, a CAT port, a PTT port, or PSKReporter/APRS spotting has since been configured (or if the spotting setting is missing - it must be explicitly off). `websdr-fldigi-start` re-applies FLDIGI's receive-only settings on every start - undoing anything changed later in FLDIGI's Configure dialogs - then checks the result and refuses to start if it isn't receive-only.
Transmit audio discarded	Each decoder's audio *output* is a virtual device that goes nowhere, so an accidental "transmit" makes no sound and reaches nothing.
No automatic transmissions	CommStat's connector is created with **RF Ack off**, so a received STATREP never triggers an over-the-air reply. FLDIGI's transmit RSID and pre-tone are off, and FLAMP's automatic and timed repeats are off.
No false spotting	JS8Call's **PSKReporter and APRS spotting**, and FLDIGI's **PSKReporter, DX-cluster, eQSL and LoTW** uploads, are all off. Stations heard through a remote receiver must never be reported as if heard at *your* location.

The programs still show their normal transmit buttons and menus; with no radio behind them they do nothing on the air.

Requirements

- **Linux with apt** and a graphical desktop session. Tested on **Linux Mint 22** (Ubuntu 24.04 base) and **EmComm Tools Community R7** (Ubuntu 22.10 base). Other Debian/Ubuntu derivatives should work but are untested.
- PipeWire or PulseAudio (the default on current Ubuntu/Mint).
- A web browser, an internet connection, and an account that can use `sudo`.
- About 1.5 GiB of free disk space (CommStat uses Qt WebEngine).
- An accurate system clock - JS8 decodes on tight time slots.

Quick start

```
git clone https://github.com/KC3WHJ/WebSDR-Digital-Receive-Only.git
cd WebSDR-Digital-Receive-Only

./install.sh --check     # prerequisite check only - changes nothing
./install.sh             # check again, ask a few questions, then install
websdr-js8-start         # JS8Call + CommStat (or "Start WebSDR JS8Call + CommStat" in your menu)
websdr-fldigi-start      # FLDIGI + FLMSG + FLAMP (or "Start WebSDR FLDIGI + FLMSG/FLAMP")
```

Don't want the FLDIGI station? `./install.sh --no-fldigi`.

You'll be asked for a callsign (or any short ID if you're only listening, such as `N0CALL`), your grid square, and optionally your US state. Then, for either station:

1. Open a web SDR or KiwiSDR in your browser and start its audio.
2. Tune it in **USB** mode - to a JS8 frequency for JS8Call (for example 7.078 MHz), or to the dial frequency you'd use on a radio for FLDIGI.
3. Press Enter in the launcher window and pick the browser stream from the menu.
4. The decoder starts working; CommStat connects to JS8Call automatically, and FLDIGI opens FLMSG and FLAMP for you.

5. When you're done: `websdr-js8-stop` / `websdr-fldigi-stop`.

Details and tuning tips: docs/USAGE.md.

A printable guide with all of this documentation: docs/WebSDR-Digital-Receive-Only-Guide.pdf (regenerate it after editing the docs with `python3 docs/build-guide.py`).

The prerequisite check

`install.sh` runs `preflight.sh` first. It is **read-only** (no changes, no sudo) and marks each item OK, WARN or FAIL. Any FAIL stops the installer before it has changed anything. It checks:

Operating system and CPU architecture - not running as root - sudo rights - free disk space and memory - Python 3.8+ and git - internet access to GitHub and to every configured apt server - that apt isn't locked by another process - that `js8call` and Qt WebEngine are actually available from your apt sources (and FLDIGI, FLMSG and FLAMP, as warnings, since that station is optional) - a reachable audio server - a graphical session and a web browser - the system clock being synchronized - that your chosen settings and ports are valid and free - and any existing install (which is kept, never overwritten).

Run `./preflight.sh --help` for its options.

What gets installed, and where

Item	Location
Packages	`js8call`, `fldigi`, `flmsg`, `flamp`, `git`, `dialog`, `pulseaudio-utils`, `python3`, `curl`, plus whatever CommStat's own installer needs. Programs you already have (e.g. EmComm Tools' own builds) are kept, not replaced.
CommStat	`~/.local/share/websdr-js8/CommStat` (cloned from GitHub)
JS8Call profile	`~/.config/JS8Call - WebSDR.ini`
FLDIGI settings	`~/.fldigi-websdr` (created at first start - a copy of `~/.fldigi` if you have one, so macros and the frequency list carry over)
FLMSG / FLAMP settings and received messages	`~/.nbems-websdr` (FLAMP in its `FLAMP/` subfolder)
Settings	`~/.config/websdr-js8/config`
Launchers	`~/.local/bin/websdr-js8-start`, `websdr-js8-stop`, `websdr-js8-watch-sink`, `websdr-fldigi-start`, `websdr-fldigi-stop`, `websdr-flmsg`, `websdr-flamp`, and menu/Desktop shortcuts
Logs	`~/.local/share/websdr-js8/js8call.log`, `commstat.log`, `routing-watcher.log`, and `fldigi/` for the FLDIGI station

Re-running the installer is safe: it keeps an existing JS8Call profile, an existing CommStat database, any connector already present, and existing FLDIGI settings.

Uninstall

```
./uninstall.sh              # removes launchers and settings; asks before deleting data
./uninstall.sh --purge      # removes the JS8Call profile, CommStat folder and FLDIGI/FLMSG/FLAMP
    folders too
```

Installed packages are left alone; remove them with `apt` if you want. Your normal `~/.fldigi` and `~/.nbems` are never touched.

Things worth knowing

- **CommStat talks to its own server.** It sends a periodic heartbeat with your callsign and grid, receives the nationwide STATREP feed, and downloads map tiles - so it needs internet. What it shows from the radio side is what the WebSDR heard, not your local RF environment.
- **CommStat updates itself** in place from its server. Don't run `git pull` inside its folder; the installer already handles this.
- **Someone else's receiver has its own rules.** Public KiwiSDRs often limit session length and users; follow the operator's terms.
- **A second, unrelated audio stream (e.g. another browser tab) won't go silent.** PipeWire/PulseAudio remembers which sink an app's audio goes to *per application*, not per tab - so without this, once the WebSDR's stream is routed in, any *other* new stream from the same browser could get silently swept in too. `websdr-js8-watch-sink` runs for the length of the session (for either station) and moves anything else that lands on the WebSDR sink back to your normal output automatically.
- **The FLDIGI station uses its own ports** (XML-RPC 7363, ARQ 7323, KISS 7344/7345) so it can run beside a normal FLDIGI (7362/7322), and FLMSG and FLAMP talk only to it. FLDIGI accepts XML-RPC connections from your local network, whatever address is configured; that's FLDIGI's own behavior.
- **RSID does the mode switching.** FLDIGI's receive RSID stays on, so it follows a net that changes modes (e.g. Contestia for the net, MFSK-32 for FLMSG/FLAMP traffic).
- **JS8Call's title bar always says "de KN4CRD".** That is the JS8Call author's callsign, built into the title of every JS8Call window (its format string is literally `%1 de KN4CRD (v%2)`). It is not your callsign and not a setting; yours is shown in the main display and used by the API and CommStat.

Testing

`tests/run-tests.sh` runs 110 checks with no sudo, GUI or internet: prerequisite failures, the installer, idempotency, launcher start/stop for both stations, audio routing and the stray-stream watcher, terminal-close survival, duplicate audio devices, the receive-only guards, FLDIGI's settings copy (rig control, PTT and spotting switched off; the normal `~/.fldigi` untouched; a fresh setup when there is none), and uninstall. It uses stand-ins for the GUIs (but CommStat's real database template and connector code) and its own profile name, ports and audio devices, so it can run beside a live install. Separate end-to-end checks ran the **real JS8Call binary** with the generated profile (it started without a wizard, answered its API, and kept the settings) and the **real FLDIGI 4.2, FLMSG 4.0 and FLAMP 2.2** (set up like an AmRRON net - 7.110 MHz, Contestia 4/250 at 900 Hz - FLDIGI decoded a recorded AmRRON training transmission played into its virtual device in full, followed it to MFSK-32 and back by RSID, and never left receive).

Tested on the air. In October 2026 the FLDIGI station monitored the live **AmRRON 20-meter net** through a WebSDR, on EmComm Tools Community R7: FLDIGI decoded the traffic, FLMSG opened a custom form and displayed its data, and FLAMP retrieved every part of a multipart message.

Not covered by the test suite: the `apt install` step and CommStat's own `linuxinstall.sh` on a clean machine (they need sudo and a fresh system). If you try it on a fresh VM or a distribution not listed above, please report what you find.

Part of a bigger station

This is the receive-only piece of a full ham radio + SDR station build for Linux Mint: **mint-sauce-for-ham** - ADS-B tracking, multi-radio selection, Winlink, VARA, WSJT-X, JS8Call, Fldigi/Flmsg and more. If you have a radio and want the whole station, start there; this repository is for anyone who only wants receive-only digital-mode listening stations.

Credits and licenses

This project only installs and configures other people's software; it does not include or redistribute it.

- **JS8Call** - Jordan Sherer (KN4CRD) and contributors, GPL-3.0.
- **CommStat** - Manuel Ochoa (N0DDK), GPL-3.0, https://github.com/mgochoa57/CommStat.
- **FLDIGI, FLMSG, FLAMP** - Dave Freese (W1HKJ), Robert Stiles (KK5VD) and contributors, GPL-3.0, http://www.w1hkj.com/.

The scripts in this repository are released under the MIT License (see `LICENSE`).

Using it

JS8Call + CommStat: a session, start to finish

1. **Start the station.** Run `websdr-js8-start` in a terminal, or use *Start WebSDR JS8Call + CommStat* from your applications menu or Desktop. It creates two virtual audio devices, starts JS8Call, waits for its API port, then starts CommStat.
2. **Play a web SDR.** In your browser, open a WebSDR (e.g. from websdr.org) or a KiwiSDR, start its audio, and tune it to a JS8 frequency in **USB** mode.
3. **Route the audio.** Back in the launcher window press Enter and choose the browser stream from the menu (it looks like *Firefox - the page title*). From now on that tab plays into JS8Call instead of your speakers - you won't hear it; that's expected.
4. **Watch it decode.** JS8Call shows decoded traffic; CommStat, which connected to JS8Call automatically, shows heard stations and any STATREPs on its map and lists.
5. **Stop.** `websdr-js8-stop` (or the *Stop* shortcut). It closes both apps and removes the virtual audio devices, so the browser plays through your speakers again.

You can close the launcher's terminal window at any time; JS8Call and CommStat keep running until you stop them.

Tuning the web SDR

JS8 is a weak-signal mode sent as audio tones, so the SDR must be in **USB** with a passband wide enough to carry roughly 500-2500 Hz of audio (a normal USB passband, about 300-2700 Hz or wider, is fine). Set the SDR's dial to the JS8 dial frequency; signals then appear at their audio offsets in JS8Call's waterfall.

JS8Call's standard dial frequencies (all USB):

Band	MHz		Band	MHz
160 m	1.842		17 m	18.104
80 m	3.578		15 m	21.078
40 m	7.078		12 m	24.922
30 m	10.130		10 m	28.078
20 m	14.078		6 m	50.318

Activity varies with the band and time of day; 40 m and 20 m are good places to start. JS8Call has no radio to tune, so the frequency it displays is only informational: set its band to match what you tuned the SDR to.

Audio level: adjust the volume on the WebSDR/KiwiSDR page (or the browser tab), not in the launcher. The level meter beside JS8Call's waterfall should sit roughly in the middle of its range - not pinned at the top, and not near zero.

CommStat

CommStat's connector is created for you (named `WebSDR`, auto-connect on, RF Ack off). The first time, you may want to:

- Use CommStat's **Manage Groups** dialog to add the groups/nets you follow, so their messages are shown.
- Optionally add a **QRZ** login (QRZ Settings) for callsign look-ups.

(These dialogs are in CommStat's menus; the exact menu layout can change between CommStat versions.)

CommStat connects to JS8Call **once, at startup**, so JS8Call has to be running first - `websdr-js8-start` takes care of that order. If you restart JS8Call on its own, open CommStat's *JS8 Connectors* dialog, select the `WebSDR` row and click **Reconnect**.

Options

Command	Effect
`websdr-js8-start --no-route`	Start the apps but skip the audio-routing step (you'll route it yourself).
`websdr-js8-start --no-commstat`	JS8Call only.
`websdr-js8-start` again	Safe at any time: everything already running is reused (handy to re-route audio after changing sites).

Changing settings

Settings live in `~/.config/websdr-js8/config` (plain `KEY=VALUE` lines): the JS8Call profile name, the virtual-device names, the JS8Call API ports, the FLDIGI station's folders, devices and ports (`FLDIGI_DIR`, `NBEMS_DIR`, `FL_SINK`, `FL_TXVOID`, `FL_XMLRPC_PORT`, `FL_ARQ_PORT`, `FL_KISS_IO_PORT`, `FL_KISS_OUT_PORT`), and your callsign and grid. Environment variables named `WEBSDR_JS8_<KEY>` override them (for example `WEBSDR_JS8_TCP_PORT=2500 websdr-js8-start`). Callsign, grid and state are also stored inside JS8Call's and CommStat's own settings; change them there.

FLDIGI + FLMSG + FLAMP: a session, start to finish

1. **Start the station.** Run `websdr-fldigi-start`, or use *Start WebSDR FLDIGI + FLMSG/FLAMP* from your applications menu or Desktop. The first time, it creates the station's own settings: a copy of `~/.fldigi` (so your macros, frequency list and modem settings carry over) in `~/.fldigi-websdr`, and FLMSG/FLAMP settings in `~/.nbems-websdr`. If you've never run FLDIGI, it starts from fresh settings with the callsign and grid you gave the installer.
2. Every start then switches off rig control, PTT and spotting, creates two virtual audio devices, and starts FLDIGI, which opens FLMSG and FLAMP.
3. **Play and tune a web SDR** in **USB** mode, to the dial frequency you'd use on a radio. Signals appear at their audio offsets in FLDIGI's waterfall (for example an AmRRON net on 7.110 MHz dial with Contestia at 900 Hz).
4. **Route the audio**: press Enter in the launcher window and pick the browser stream.
5. **Watch it decode.** Click a signal in the waterfall, or let RSID switch the mode for you. Received FLMSG forms open in FLMSG; FLAMP collects received files in `~/.nbems-websdr/FLAMP/rx`.
6. **Stop.** `websdr-fldigi-stop` closes FLAMP, FLMSG and FLDIGI (asking FLDIGI to close itself, so it saves its settings) and removes its virtual audio devices.

Both stations can run at the same time: they use different audio devices and ports. Play each in its own browser tab (one per frequency) and pick the right tab in each launcher. The FLDIGI station can also run beside a normal, radio-connected FLDIGI.

Command	Effect
`websdr-fldigi-start --no-route`	Start FLDIGI, FLMSG and FLAMP but skip the audio-routing step.
`websdr-fldigi-start` again	Safe at any time: re-routes the audio and reuses everything already running.

How it works

The audio path

```
browser tab (WebSDR / KiwiSDR audio)
       |  routed by websdr-js8-start   (pactl move-sink-input)
       v
virtual device  "websdr_sink"        <- a PulseAudio/PipeWire "null sink": audio goes in,
       |                                nothing plays, and its monitor can be recorded
       v   ("websdr_sink.monitor" is what JS8Call is set to listen to)
     JS8Call  (its own profile: js8call -r WebSDR)
       |  TCP API on 127.0.0.1:2443  (one client allowed by JS8Call)
       v
     CommStat  (one connector, "WebSDR", to that port)

JS8Call's *output* -> virtual device "websdr_txvoid"  (a second null sink: transmit audio is
   discarded)
```

Both virtual devices are created on start and removed on stop. The desktop's own audio is never touched: only the one browser stream you pick is moved.

Why a separate JS8Call profile

`js8call -r WebSDR` uses its own settings file (`~/.config/JS8Call - WebSDR.ini`), data folder and API port. A normal JS8Call configured for a real radio is unaffected by this station and vice versa, and both can run at once.

The receive-only guard

The profile is written with no rig control. Because a profile can be edited later from inside JS8Call (Settings > Radio), `websdr-js8-start` re-reads it on every start and refuses to launch if `Rig` isn't `None`, if a CAT serial port, CAT network port or PTT port is set, or if `PSKReporter` / `SpotToAPRS` isn't explicitly `false` (a remote receiver's decodes must never be reported under your callsign and grid). It tells you exactly what to fix. This is a check on this station's own configuration, not a claim about the programs: JS8Call and CommStat are unmodified.

Startup order and detaching

CommStat connects to JS8Call once at startup and gives up (and disables the connector) after repeated failures, so the launcher starts JS8Call, **waits for its API port to open**, and only then starts CommStat.

Both programs are started with `setsid -f nohup ...`: forked into their own session right away, so they have no terminal to hang up. Two things learned the hard way:

- A background job started with a plain `&` (even with `disown`) still receives a hang-up when its terminal window closes.
- `nohup` alone protects a program but **not** helper processes it starts. CommStat's Qt WebEngine helper processes (which draw the map) die on the hang-up, leaving CommStat running with a blank map. A fully separate session avoids both.

Virtual devices: never duplicated, always fully removed

Existence is checked in the *module* list (accurate the instant a load returns) rather than the sink list, so two quick starts can't create a duplicate. Removal unloads every matching module one call at a time, because passing two module IDs to one `pactl unload-module` call fails and would strand a silent device. A stranded device matters: PipeWire remembers where a browser's audio was routed, so a browser can end up playing into a device that goes nowhere.

The installer

1. **Prerequisite check** (read-only). A FAIL stops here with nothing changed.
2. Your callsign, grid and state (flags or prompts; validated before anything is installed).
3. `apt` packages.
4. CommStat: cloned, then its own `linuxinstall.sh` installs Qt WebEngine and the Python packages. An existing CommStat is reused; it is only `git pull`ed if its checkout is clean, since CommStat rewrites its own files when its server pushes an update.
5. `share/seed_commstat.py` creates CommStat's database from its own shipped template, fills in your details **only where still blank**, and adds the single `WebSDR` connector through CommStat's own `ConnectorManager` (so validation and defaults are exactly what its dialog would produce), with auto-connect on and RF Ack off.
6. The JS8Call profile is written from `share/JS8Call-profile.ini.template` **only if it doesn't exist**.
7. Launchers, shortcuts and settings are installed, then the result is verified.

Every step is idempotent: re-running keeps existing profiles, databases and connectors.

Layout

```
install.sh  uninstall.sh  preflight.sh
lib/common.sh               settings, validation, audio-device helpers (sourced by everything)
bin/websdr-js8-start        JS8Call + CommStat launcher
bin/websdr-js8-stop         its stopper
bin/websdr-js8-watch-sink   stray-audio watcher (used by both stations)
bin/websdr-fldigi-start     FLDIGI + FLMSG + FLAMP launcher
bin/websdr-fldigi-stop      its stopper
bin/websdr-flmsg            FLMSG / FLAMP with the station's own folder and ports
bin/websdr-flamp              (FLDIGI starts these)
share/JS8Call-profile.ini.template
share/seed_commstat.py      CommStat first-run configuration
tests/run-tests.sh          110 checks; tests/stubs/ holds the JS8Call, FLDIGI, FLMSG and FLAMP stand-
    ins
```

The FLDIGI station

```
browser tab (WebSDR / KiwiSDR audio)
       |  routed by websdr-fldigi-start
       v
virtual device  "websdr_fldigi"           (FLDIGI listens to "websdr_fldigi.monitor")
       v
     FLDIGI  (fldigi --config-dir ~/.fldigi-websdr/ --flmsg-dir ~/.nbems-websdr/,
       |      XML-RPC 7363, ARQ 7323, KISS 7344/7345)
       |-- XML-RPC --> FLMSG  (flmsg --flmsg-dir ~/.nbems-websdr/)
       '-- XML-RPC + ARQ --> FLAMP  (flamp --config-dir ~/.nbems-websdr/)

FLDIGI's *output* -> virtual device "websdr_fldigi_txvoid"  (transmit audio is discarded)
```

- **Audio**: FLDIGI is set to its PulseAudio back end and started with `PULSE_SOURCE=websdr_fldigi.monitor PULSE_SINK=websdr_fldigi_txvoid`, which picks its input and output without depending on device numbers.
- **Its own devices, not the JS8Call station's**: `websdr-js8-stop` removes its devices, which would cut off FLDIGI's audio if they were shared.
- **Settings are re-applied, not just checked.** FLDIGI rewrites `fldigi_def.xml` when it exits, so the launcher edits it only while FLDIGI is stopped: every start switches off rig control (Hamlib, RigCAT, flrig), every PTT method, PSKReporter/DX-cluster/eQSL/LoTW, transmit RSID and pre-tone; sets its own ports; and points FLDIGI's autostart at the `websdr-flmsg` / `websdr-flamp` wrappers. It then re-reads the file and refuses to start unless the result is receive-only. Settings missing from the file are added, so nothing is left to a default.
- **A copy, never the original**: the first start copies `~/.fldigi` and rewrites the copy's absolute paths into `~/.fldigi` (such as the macro file) to point at the copy. `~/.fldigi` and `~/.nbems` are only ever read.
- **Stopping**: FLAMP and FLMSG are closed first (FLAMP can hang if FLDIGI goes first). FLDIGI is asked to close over XML-RPC (`fldigi.terminate`), because a signal makes it exit without saving its settings; a signal is only the fallback.
- **FLAMP's option name**: its `--help` says `--confg-dir`, but the option FLAMP accepts is `--config-dir`, and it takes the NBEMS folder (FLAMP adds `FLAMP/` itself).

Troubleshooting

Logs: `~/.local/share/websdr-js8/js8call.log`, `commstat.log`, and `fldigi/fldigi.log`. Run `./preflight.sh` any time for a read-only health check of the machine.

The installer stops with FAIL items

Each FAIL line ends with a hint. Common ones:

FAIL	Meaning / fix
Package manager busy	Another apt/dpkg process holds the lock. Linux Mint's background refresh does this for a minute or two after boot; wait and run again.
package: js8call not available	Enable the `universe` component (Ubuntu) or check your Mint sources, then `sudo apt update`.
package: python3-pyqt5.qtwebengine not available, on an end-of-life Ubuntu (e.g. 22.10, which EmComm Tools R7 uses)	The release's packages have moved to `old-releases.ubuntu.com`. Point apt there: `sudo sed -i -E 's#http://(us\.archive|security)\.ubuntu\.com#http://old-releases.ubuntu.com#' /etc/apt/sources.list && sudo apt update` (back up `sources.list` first).
Audio server	`pactl` can't reach PulseAudio/PipeWire - run from a terminal inside your logged-in desktop, not plain ssh.
Desktop session	No `DISPLAY`/`WAYLAND_DISPLAY`: JS8Call and CommStat are desktop apps.
TCP port in use	Something else is listening on JS8Call's API port. Set another `TCP_PORT` in `~/.config/websdr-js8/config`.
Running as root	Run as your normal user; the installer uses sudo itself where needed.

Nothing is installed or changed when a FAIL stops the installer.

JS8Call shows nothing / no audio

- Is the WebSDR page actually **playing**? Many sites need a click on a play/"start audio" button.
- Was the stream routed? Run `websdr-js8-start` again after the audio is playing (safe to repeat).
- With the stream routed you will **not** hear it from your speakers; that's expected.
- Check the SDR is in **USB** mode and tuned to the JS8 dial frequency (see USAGE.md).
- Check your clock: `timedatectl` should say "System clock synchronized: yes".
- Weak or crowded band? Try 40 m or 20 m at a busy time of day.

The browser goes silent after I finish

Run `websdr-js8-stop`. It removes the virtual devices so the browser's audio returns to your speakers. (If a page is still silent afterwards, reload it.)

CommStat can't connect / its connector says Disabled

CommStat connects to JS8Call once at startup and disables the connector after repeated failures. Make sure JS8Call is running, then in CommStat open **JS8 Connectors**, select `WebSDR` and click **Reconnect**. `websdr-js8-start` always starts JS8Call first.

CommStat's map is blank

Its map is drawn by helper processes that must survive the launcher's terminal closing; `websdr-js8-start` starts CommStat fully detached so they do. If you launched CommStat some other way and the map is blank, quit it and use `websdr-js8-start`. The map also needs internet access for its tiles.

"receive-only guard: ..." and it won't start

The JS8Call profile no longer looks receive-only (rig control, a CAT port or a PTT port is set - possibly changed from inside JS8Call). The message says which. Fix it in `~/.config/JS8Call - WebSDR.ini`, or remove that file and re-run `./install.sh` for a fresh profile.

The JS8Call window title says "de KN4CRD"

That is normal for every JS8Call: KN4CRD is the JS8Call author's callsign, built into the window-title format (`... de KN4CRD (v2.x)`). It isn't your callsign and can't be changed. Your own callsign is shown in the main display (under the frequency) and is what JS8Call's API and CommStat use.

An old install left a silent audio device behind

`websdr-js8-stop` removes every copy of its two virtual devices. To check: `pactl list short modules | grep websdr`.

Updating

- **CommStat** updates itself when its server pushes an update. Don't `git pull` in its folder.
- **JS8Call** updates through apt like any package.
- **This project:** `git pull` here, then `./install.sh` again (it keeps your existing configuration).

FLDIGI shows nothing / a flat waterfall

- Same first checks as for JS8Call: is the WebSDR page **playing**, was the stream routed (run `websdr-fldigi-start` again), and is the SDR in **USB**?
- The two stations have separate audio devices. A tab routed into JS8Call doesn't reach FLDIGI; play a second tab for FLDIGI and pick that one.
- Check FLDIGI is on PulseAudio: *Configure* > *Sound Card* > *Audio devices*. The launcher sets this at every start; if it was changed while FLDIGI was running, restart the station.

FLMSG or FLAMP doesn't open, or says it can't reach FLDIGI

They are started by FLDIGI (*Configure* > *Misc* > *Autostart*), through `websdr-flmsg` and `websdr-flamp`, and connect to this station's FLDIGI on port 7363 (not the usual 7362). Stop and start the station to re-apply that. A FLMSG or FLAMP you open some other way connects to a normal FLDIGI instead.

FLDIGI's settings changes don't stick

Rig control, PTT, spotting, ports and audio settings are re-applied at every start, by design. Other changes (modem settings, colors, macros) are kept, provided FLDIGI is closed with `websdr-fldigi-stop` or its own File > Exit.